

PMSCS 694

Information Security

Crypto- Symmetric Key

Lecture 4

Md. Rafsan Jani
Associate Professor
Department of Computer Science and Engineering
Jahangirnagar University

Symmetric Key Crypto

The chief forms of beauty are order and symmetry...

—Aristotle

“You boil it in sawdust: you salt it in glue:
You condense it with locusts and tape:
Still keeping one principal object in view —
To preserve its symmetrical shape.”

—Lewis Carroll, *The Hunting of the Snark*

Symmetric Key Crypto

- ❑ Stream cipher —generalize one-time pad
 - Except that key is relatively short
 - Key is stretched into a long **keystream**
 - Keystream is used just like a one-time pad
- ❑ Block cipher —generalized codebook
 - Block cipher **key** determines a codebook
 - Each key yields a different codebook
 - Employs both “confusion” and “diffusion”

Stream Ciphers



Stream Ciphers

- ❑ Once upon a time, not so very long ago... stream ciphers were the king of crypto
- ❑ Today, not as popular as block ciphers
- ❑ We'll discuss two stream ciphers:
- ❑ A5/1
 - Based on shift registers
 - Used in GSM mobile phone system
- ❑ RC4
 - Based on a changing lookup table
 - Used many places

A5/1 Naming Breakdown

- "A5":
 - The "A" is simply part of the designation used by the GSM standard to refer to the encryption algorithms used to secure communication between mobile devices and base stations.
 - "5" refers to the fact that **A5/1** is the **first version** of the **A5 encryption algorithm** used in GSM.
- "/1":
 - The "/1" is used to differentiate between **two versions** of the A5 algorithm: **A5/1** and **A5/2**.
 - **A5/1** is the stronger of the two and was intended for use in more secure regions.
 - **A5/2**, on the other hand, was a **weaker version** that was designed to comply with export regulations, especially in countries with stricter cryptographic restrictions.

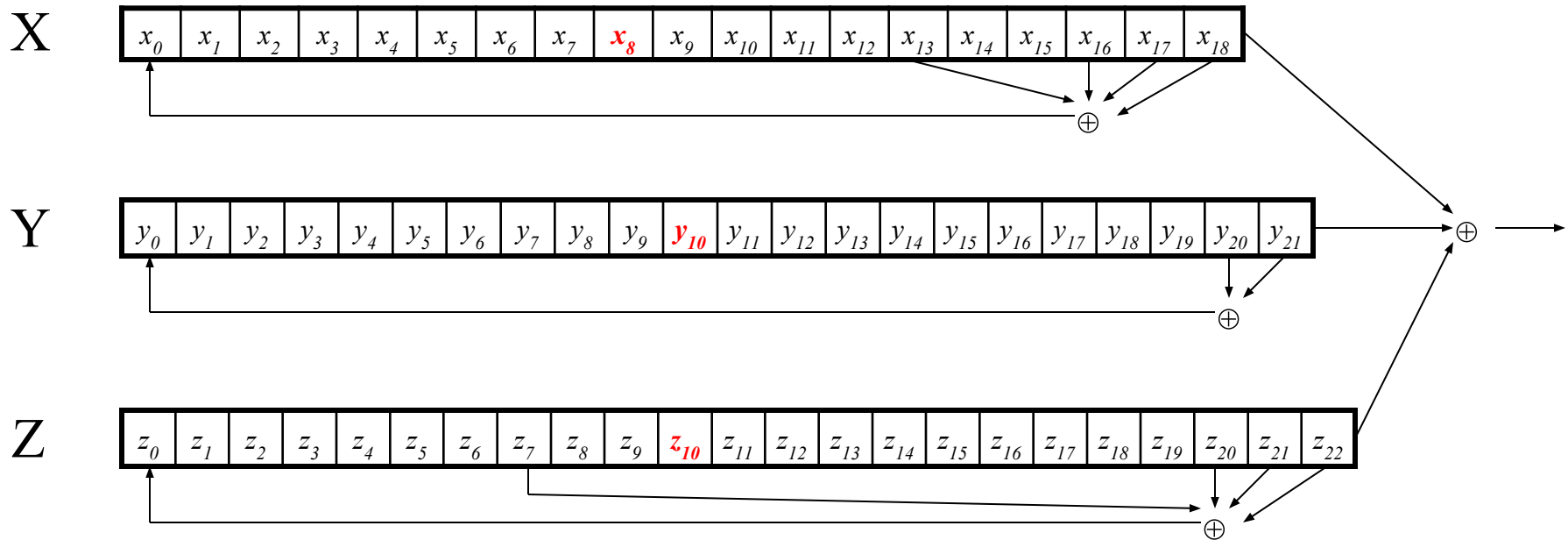
A5/1: Shift Registers

- A5/1 uses 3 *shift registers*
 - X: 19 bits ($x_0, x_1, x_2, \dots, x_{18}$)
 - Y: 22 bits ($y_0, y_1, y_2, \dots, y_{21}$)
 - Z: 23 bits ($z_0, z_1, z_2, \dots, z_{22}$)

A5/1: Keystream

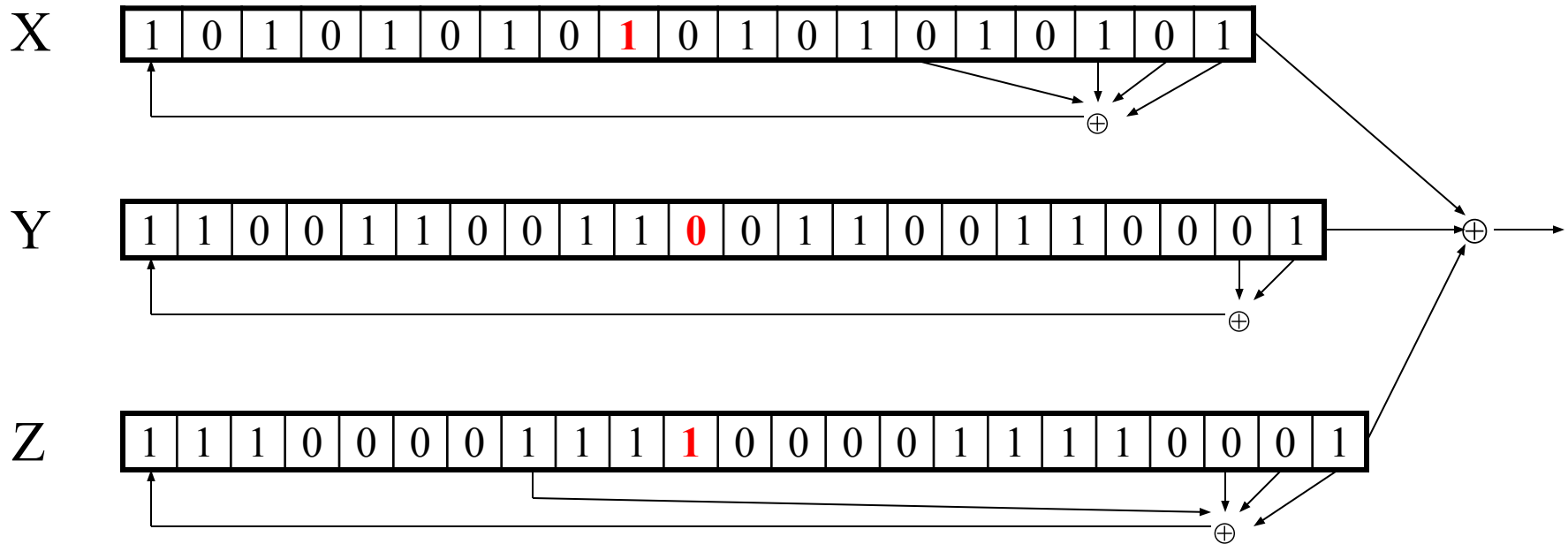
- At each iteration: $m = \text{maj}(x_8, y_{10}, z_{10})$
 - Examples: $\text{maj}(0,1,0) = 0$ and $\text{maj}(1,1,0) = 1$
- If $x_8 = m$ then *X steps*
 - $t = x_{13} \oplus x_{16} \oplus x_{17} \oplus x_{18}$
 - $x_i = x_{i-1}$ for $i = 18, 17, \dots, 1$ and $x_0 = t$
- If $y_{10} = m$ then *Y steps*
 - $t = y_{20} \oplus y_{21}$
 - $y_i = y_{i-1}$ for $i = 21, 20, \dots, 1$ and $y_0 = t$
- If $z_{10} = m$ then *Z steps*
 - $t = z_7 \oplus z_{20} \oplus z_{21} \oplus z_{22}$
 - $z_i = z_{i-1}$ for $i = 22, 21, \dots, 1$ and $z_0 = t$
- Keystream **bit** is $x_{18} \oplus y_{21} \oplus z_{22}$

A5/1



- ❑ Each variable here is a single bit
- ❑ Key is used as **initial fill** of registers
- ❑ Each register steps (or not) based on $\text{maj}(x_8, y_{10}, z_{10})$
- ❑ Keystream bit is XOR of rightmost bits of registers

A5/1



- ❑ In this example, $m = \text{maj}(x_8, y_{10}, z_{10}) = \text{maj}(\mathbf{1}, \mathbf{0}, \mathbf{1}) = \mathbf{1}$
- ❑ Register X steps, Y does not step, and Z steps
- ❑ Keystream bit is XOR of right bits of registers
- ❑ Here, keystream bit will be $0 \oplus 1 \oplus 0 = 1$

Shift Register Crypto

- ❑ Shift register crypto efficient in hardware
- ❑ Often, slow if implemented in software
- ❑ In the past, very, very popular
- ❑ Today, more is done in software due to fast processors
- ❑ Shift register crypto still used some
 - Especially in resource-constrained devices

RC4

- ❑ RC4 (Rivest Cipher 4) is a **stream cipher** designed by **Ron Rivest** in 1987.
- ❑ It's known for its **simplicity** and **speed**, making it widely used in **network protocols** and **applications**, such as **SSL/TLS** and **WEP** (Wi-Fi security) for many years.

RC4

- ❑ A self-modifying lookup table
- ❑ Table always contains a permutation of the byte values $0, 1, \dots, 255$
- ❑ Initialize the permutation using key
- ❑ At each step, RC4 does the following
 - Swaps elements in current lookup table
 - Selects a keystream byte from table
- ❑ Each step of RC4 produces a **byte**
 - Efficient in software
- ❑ Each step of A5/1 produces only a **bit**
 - Efficient in hardware

RC4 Initialization

- `S[]` is permutation of `0,1,...,255`
- `key[]` contains `N` bytes of key

```
for i = 0 to 255
    S[i] = i
    K[i] = key[i (mod N)]
next i
j = 0
for i = 0 to 255
    j = (j + S[i] + K[i]) mod 256
    swap(S[i], S[j])
next i
i = j = 0
```

RC4 Initialization Example

- Let's take a 3-byte key:
 - - Key = $[K1, K2, K3] = [1, 2, 3]$
- *Initialize the state array:*
 - $S = [0, 1, 2, 3, 4, 5, \dots, 255]$
- *Set $j = 0$.*
 - *Now perform the key expansion:*
 - *For $i = 0$, we take the first byte of the key $K1 = 1$.*
 - $j = (j + S[0] + K1) \bmod 256 = (0 + 0 + 1) \bmod 256 = 1$
 - *Swap $S[0]$ and $S[1]$:*
 - $S = [1, 0, 2, 3, 4, 5, \dots]$
- *For $i = 1$, we take the second byte of the key $K2 = 2$.*
 - $j = (j + S[1] + K2) \bmod 256 = (1 + 0 + 2) \bmod 256 = 3$
 - *Swap $S[1]$ and $S[3]$:*
 - $S = [1, 3, 2, 0, 4, 5, \dots]$
- *For $i = 2$, we take the third byte of the key $K3 = 3$.*
 - $j = (j + S[2] + K3) \bmod 256 = (3 + 2 + 3) \bmod 256 = 8$
 - *Swap $S[2]$ and $S[8]$:*
 - $S = [1, 3, 4, 0, 5, 6, 7, 8, 2, 9, \dots]$
- *This completes the KSA for a 3-byte key.*
- *The final state array S is now a scrambled version based on the key.*

RC4 Keystream

- At each step, swap elements in table and select keystream byte

Increment i: $i = (i + 1) \bmod 256$

Update j: $j = (j + S[i]) \bmod 256$

Swap: $\text{swap}(S[i], S[j])$

Get Index t: $t = (S[i] + S[j]) \bmod 256$

Generate key byte: $\text{keystreamByte} = S[t]$

- Use keystream bytes like a one-time pad
- **Note:** first 256 bytes should be discarded
 - Otherwise, related key attack exists

RC4 Keystream Example

- ❑ Start with $i = 0, j = 0$.
- ❑ $i = 1, j = (0 + S[0]) \bmod 256 = 1$
 - Swap $S[0]$ and $S[1]$
 - Keystream byte: $K = S[(S[1] + S[1]) \bmod 256] = S[2]$
- ❑ $i = 2, j = (1 + S[1]) \bmod 256 = 3$
 - Swap $S[2]$ and $S[3]$
 - Keystream byte: $K = S[(S[2] + S[3]) \bmod 256]$
- ❑ Repeat this process to generate more keystream bytes.

Stream Ciphers

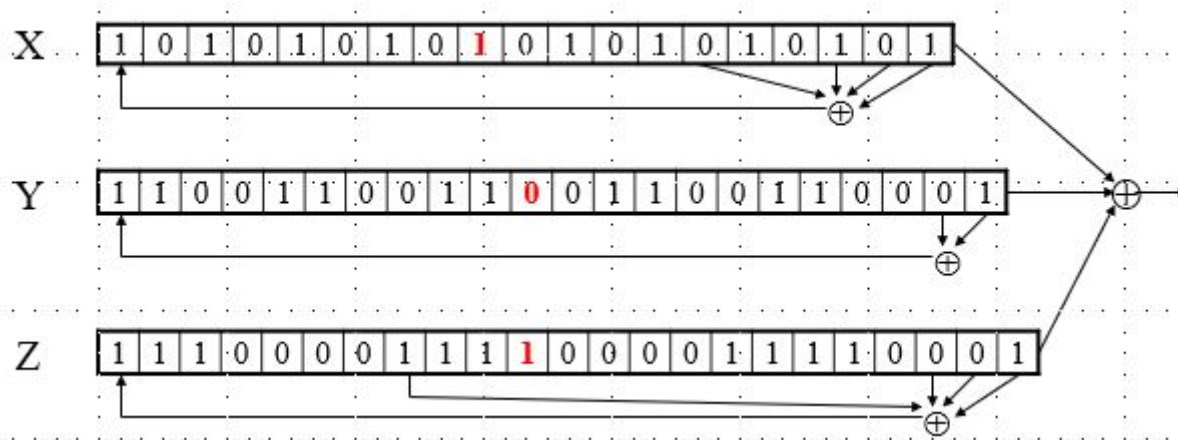
- ❑ Stream ciphers were popular in the past
 - Efficient in hardware (shift register types)
 - Speed was needed to keep up with voice, etc.
 - Today, processors are fast, so software-based crypto is usually more than fast enough
- ❑ Future of stream ciphers?
 - Shamir declared "the death of stream ciphers"
 - May be greatly exaggerated...

Sample Questions

- ❑ What is a stream cipher? Provide examples of situations where it is used.
- ❑ Explain the difference between stream ciphers and block ciphers.
- ❑ How does the A5/1 stream cipher generate its keystream? Include a brief explanation of its shift registers.
- ❑ Write down the pseudocode for the RC4 Initialization phase.

Sample Questions

- The A5/1 cipher uses three shift registers X,Y,Z with lengths of 19, 22, and 23 bits, respectively.



- Suppose the majority function m at a specific time is computed as: $m = \text{maj}(X[8], Y[10], Z[10])$. Given the initial states: $X[8]=1$, $Y[10]=0$, $Z[10]=1$.
- Compute the value of m , determine which registers will step, and calculate the new state of X if its feedback polynomial is: $t = X[13] \oplus X[16] \oplus X[17] \oplus X[18]$

Sample Questions

- For A5/1 answer the followings:
 - ✓ On average, how often does the X register step?
 - ✓ On average, how often does the Y register step?
 - ✓ On average, how often does the Z register step?
 - ✓ On average, how often do all three registers step?
 - ✓ On average, how often do exactly two registers step?
 - ✓ On average, how often does exactly one register step?

Sample Questions

You are given the following initial state of the registers:

- $X = 1011010110101001101$ (19 bits)
- $Y = 1100101101001101001011$ (22 bits)
- $Z = 01101001110010110101101$ (23 bits)

□ Now Answer the followings

- Compute the majority clocking bit for the first cycle (i.e., the value of $\text{maj}(X[8], Y[10], Z[10])$).
- Which registers will step in the first cycle?
- Using the initial state, calculate the keystream bit (k_1) produced in the first clock cycle.
- Compute the feedback bits for both the X and Z registers, based on the given initial state. Show the steps for calculating the feedback bits.
- What is the new majority clocking bit for the second cycle (i.e., $\text{maj}(X[8], Y[10], Z[10])$)?
- Which registers will step in the second cycle, and why?
- Calculate the keystream bit produced in the second cycle.
- Provide a step-by-step explanation of how the registers update after the second cycle. Include the feedback bits and the new states of the registers (X, Y, and Z)